

3.4 迭代器介绍



106

我们已经知道可以使用下标运算符来访问 `string` 对象的字符或 `vector` 对象的元素，还有另外一种更通用的机制也可以实现同样的目的，这就是**迭代器**（`iterator`）。在第 II 部分中将要介绍，除了 `vector` 之外，标准库还定义了其他几种容器。所有标准库容器都可以使用迭代器，但是其中只有少数几种才同时支持下标运算符。严格来说，`string` 对象不属于容器类型，但是 `string` 支持很多与容器类型类似的操作。`vector` 支持下标运算符，这点和 `string` 一样；`string` 支持迭代器，这也和 `vector` 是一样的。

类似于指针类型（参见 2.3.2 节，第 47 页），迭代器也提供了对对象的间接访问。就迭代器而言，其对象是容器中的元素或者 `string` 对象中的字符。使用迭代器可以访问某个元素，迭代器也能从一个元素移动到另外一个元素。迭代器有有效和无效之分，这一点和指针差不多。有效的迭代器或者指向某个元素，或者指向容器中尾元素的下一位置；其他所有情况都属于无效。

3.4.1 使用迭代器



和指针不一样的是，获取迭代器不是使用取地址符，有迭代器的类型同时拥有返回迭代器的成员。比如，这些类型都拥有名为 `begin` 和 `end` 的成员，其中 `begin` 成员负责返回指向第一个元素（或第一个字符）的迭代器。如有下述语句：

```
// 由编译器决定 b 和 e 的类型；参见 2.5.2 节（第 61 页）
// b 表示 v 的第一个元素，e 表示 v 尾元素的下一位置
auto b = v.begin(), e = v.end(); // b 和 e 的类型相同
```

`end` 成员则负责返回指向容器（或 `string` 对象）“尾元素的下一位置（one past the end）”的迭代器，也就是说，该迭代器指示的是容器的一个本不存在的“**尾后**（off the end）”元素。这样的迭代器没什么实际含义，仅是个标记而已，表示我们已经处理完了容器中的所有元素。`end` 成员返回的迭代器常被称作**尾后迭代器**（off-the-end iterator）或者简称为尾迭代器（end iterator）。特殊情况下如果容器为空，则 `begin` 和 `end` 返回的是同一个迭代器。



如果容器为空，则 `begin` 和 `end` 返回的是同一个迭代器，都是尾后迭代器。

一般来说，我们不清楚（不在意）迭代器准确的类型到底是什么。在上面的例子中，使用 `auto` 关键字定义变量 `b` 和 `e`（参见 2.5.2 节，第 61 页），这两个变量的类型也就是 `begin` 和 `end` 的返回值类型，第 97 页将对相关内容做更详细的介绍。

迭代器运算符

表 3.6 列举了迭代器支持的一些运算。使用 `==` 和 `!=` 来比较两个合法的迭代器是否相等，如果两个迭代器指向的元素相同或者都是同一个容器的尾后迭代器，则它们相等；否则就说这两个迭代器不相等。

表 3.6: 标准容器迭代器的运算符	
*iter	返回迭代器 iter 所指元素的引用
iter->mem	解引用 iter 并获取该元素的名为 mem 的成员, 等价于 (*iter).mem
++iter	令 iter 指示容器中的下一个元素
--iter	令 iter 指示容器中的上一个元素
iter1 == iter2	判断两个迭代器是否相等 (不相等), 如果两个迭代器指示的是同一个元
iter1 != iter2	素或者它们是同一个容器的尾后迭代器, 则相等; 反之, 不相等

107 和指针类似, 也能通过解引用迭代器来获取它所指示的元素, 执行解引用的迭代器必须合法并确实指示着某个元素 (参见 2.3.2 节, 第 48 页)。试图解引用一个非法迭代器或者尾后迭代器都是未被定义的行为。

举个例子, 3.2.3 节 (第 84 页) 中的程序利用下标运算符把 string 对象的第一个字母改为了大写形式, 下面利用迭代器实现同样的功能:

```
string s("some string");
if (s.begin() != s.end()) { // 确保 s 非空
    auto it = s.begin();    // it 表示 s 的第一个字符
    *it = toupper(*it);    // 将当前字符改成大写形式
}
```

本例和原来的程序一样, 首先检查 s 是否为空, 显然通过检查 begin 和 end 返回的结果是否一致就能做到这一点。如果返回的结果一样, 说明 s 为空; 如果返回的结果不一样, 说明 s 不为空, 此时 s 中至少包含一个字符。

我们在 if 内部, 声明了一个迭代器变量 it 并把 begin 返回的结果赋给它, 这样就得到了指示 s 中第一个字符的迭代器, 接下来通过解引用运算符将第一个字符更改为大写形式。和原来的程序一样, 输出结果将是:

Some string

将迭代器从一个元素移动到另外一个元素

迭代器使用递增 (++) 运算符 (参见 1.4.1 节, 第 11 页) 来从一个元素移动到下一个元素。从逻辑上来说, 迭代器的递增和整数的递增类似, 整数的递增是在整数值上“加 1”, 迭代器的递增则是将迭代器“向前移动一个位置”。



因为 end 返回的迭代器并不实际指示某个元素, 所以不能对其进行递增或解引用的操作。

之前有一个程序把 string 对象中第一个单词改写为大写形式, 现在利用迭代器及其递增运算符可以实现相同的功能:

```
108 // 依次处理 s 的字符直至我们处理完全部字符或者遇到空白
for (auto it = s.begin(); it != s.end() && !isspace(*it); ++it)
    *it = toupper(*it); // 将当前字符改成大写形式
```

和 3.2.3 节 (第 84 页) 的那个程序一样, 上面的循环也是遍历 s 的字符直到遇到空白字符为止, 只不过之前的程序用的是下标运算符, 现在这个程序用的是迭代器。

循环首先用 s.begin 的返回值来初始化 it, 意味着 it 指示的是 s 中的第一个字符 (如果有的话)。条件部分检查是否已到达 s 的尾部, 如果尚未到达, 则将 it 解引用的结

果传入 `isspace` 函数检查是否遇到了空白。每次迭代的最后，执行 `++it` 令迭代器前移一个位置以访问 `s` 的下一个字符。

循环体内部和上一个程序 `if` 语句内的最后一句话一样，先解引用 `it`，然后将结果传入 `toupper` 函数得到该字母对应的大写形式，再把这个大写字母重新赋值给 `it` 所指示的字符。

关键概念：泛型编程

原来使用 C 或 Java 的程序员在转而使用 C++ 语言之后，会对 `for` 循环中使用 `!=` 而非 `<` 进行判断有点儿奇怪，比如上面的这个程序以及 85 页的那个。C++ 程序员习惯性地使用 `!=`，其原因和他们更愿意使用迭代器而非下标的原因一样：因为这种编程风格在标准库提供的所有容器上都有效。

之前已经说过，只有 `string` 和 `vector` 等一些标准库类型有下标运算符，而并非全都如此。与之类似，所有标准库容器的迭代器都定义了 `==` 和 `!=`，但是它们中的大多数都没有定义 `<` 运算符。因此，只要我们养成使用迭代器和 `!=` 的习惯，就不用太在意用的到底是哪种容器类型。

迭代器类型

就像不知道 `string` 和 `vector` 的 `size_type` 成员（参见 3.2.2 节，第 79 页）到底是什么类型一样，一般来说我们也不知道（其实是无须知道）迭代器的精确类型。而实际上，那些拥有迭代器的标准库类型使用 `iterator` 和 `const_iterator` 来表示迭代器的类型：

```
vector<int>::iterator it;      // it 能读写 vector<int> 的元素
string::iterator it2;        // it2 能读写 string 对象中的字符

vector<int>::const_iterator it3; // it3 只能读元素，不能写元素
string::const_iterator it4;      // it4 只能读字符，不能写字符
```

`const_iterator` 和常量指针（参见 2.4.2 节，第 56 页）差不多，能读取但不能修改它所指的元素值。相反，`iterator` 的对象可读可写。如果 `vector` 对象或 `string` 对象是一个常量，只能使用 `const_iterator`；如果 `vector` 对象或 `string` 对象不是常量，那么既能使用 `iterator` 也能使用 `const_iterator`。

术语：迭代器和迭代器类型

迭代器这个名词有三种不同的含义：可能是迭代器概念本身，也可能是指容器定义的迭代器类型，还可能是指某个迭代器对象。

重点是理解存在一组概念上相关的类型，我们认定某个类型是迭代器当且仅当它支持一套操作，这套操作使得我们能访问容器的元素或者从某个元素移动到另外一个元素。

每个容器类定义了一个名为 `iterator` 的类型，该类型支持迭代器概念所规定的一套操作。

`begin` 和 `end` 运算符

`begin` 和 `end` 返回的具体类型由对象是否是常量决定，如果对象是常量，`begin` 和 `end` 返回 `const_iterator`；如果对象不是常量，返回 `iterator`：

```
vector<int> v;
```

```
const vector<int> cv;
auto it1 = v.begin();    // it1 的类型是 vector<int>::iterator
auto it2 = cv.begin();   // it2 的类型是 vector<int>::const_iterator
```

有时候这种默认的行为并非我们所要。在 6.2.3 节（第 191 页）中将会看到，如果对象只需读操作而无须写操作的话最好使用常量类型（比如 `const_iterator`）。为了便于专门得到 `const_iterator` 类型的返回值，C++11 新标准引入了两个新函数，分别是 `cbegin` 和 `cend`：

C++
11

```
auto it3 = v.cbegin(); // it3 的类型是 vector<int>::const_iterator
```

类似于 `begin` 和 `end`，上述两个新函数也分别返回指示容器第一个元素或最后元素下一位置的迭代器。有所不同的是，不论 `vector` 对象（或 `string` 对象）本身是否是常量，返回值都是 `const_iterator`。

结合解引用和成员访问操作

解引用迭代器可获得迭代器所指的对象，如果该对象的类型恰好是类，就有可能希望进一步访问它的成员。例如，对于一个由字符串组成的 `vector` 对象来说，要想检查其元素是否为空，令 `it` 是该 `vector` 对象的迭代器，只需检查 `it` 所指字符串是否为空就可以了，其代码如下所示：

```
(*it).empty()
```

注意，`(*it).empty()` 中的圆括号必不可少，具体原因将在 4.1.2 节（第 121 页）介绍，该表达式的含义是先对 `it` 解引用，然后解引用的结果再执行点运算符（参见 1.5.2 节，第 20 页）。如果不加圆括号，点运算符将由 `it` 来执行，而非 `it` 解引用的结果：

```
(*it).empty()    // 解引用 it，然后调用结果对象的 empty 成员
*it.empty()       // 错误：试图访问 it 的名为 empty 的成员，但 it 是个迭代器，
                  // 没有 empty 成员
```

110

上面第二个表达式的含义是从名为 `it` 的对象中寻找其 `empty` 成员，显然 `it` 是一个迭代器，它没有哪个成员是叫 `empty` 的，所以第二个表达式将发生错误。

为了简化上述表达式，C++ 语言定义了**箭头运算符**（`->`）。箭头运算符把解引用和成员访问两个操作结合在一起，也就是说，`it->mem` 和 `(*it).mem` 表达的意思相同。

例如，假设用一个名为 `text` 的字符串向量存放文本文件中的数据，其中的元素或者是一句话或者是一个用于表示段落分隔的空字符串。如果要输出 `text` 中第一段的内容，可以利用迭代器写一个循环令其遍历 `text`，直到遇到空字符串的元素为止：

```
// 依次输出 text 的每一行直至遇到第一个空白行为止
for (auto it = text.cbegin();
     it != text.cend() && !it->empty(); ++it)
    cout << *it << endl;
```

我们首先初始化 `it` 令其指向 `text` 的第一个元素，循环重复执行直至处理完了 `text` 的所有元素或者发现某个元素为空。每次迭代时只要发现还有元素并且尚未遇到空元素，就输出当前正在处理的元素。值得注意的是，因为循环从头到尾只是读取 `text` 的元素而未向其中写值，所以使用了 `cbegin` 和 `cend` 来控制整个迭代过程。

某些对 `vector` 对象的操作会使迭代器失效

3.3.2 节（第 90 页）曾经介绍过，虽然 `vector` 对象可以动态地增长，但是也会有一

些副作用。已知的一个限制是不能在范围 for 循环中向 vector 对象添加元素。另外一个限制是任何一种可能改变 vector 对象容量的操作，比如 push_back，都会使该 vector 对象的迭代器失效。9.3.6 节（第 315 页）将详细解释迭代器是如何失效的。



WARNING

谨记，但凡是使用了迭代器的循环体，都不要向迭代器所属的容器添加元素。

3.4.1 节练习

练习 3.21: 请使用迭代器重做 3.3.3 节（第 94 页）的第一个练习。

练习 3.22: 修改之前那个输出 text 第一段的程序，首先把 text 的第一段全都改成大写形式，然后再输出它。

练习 3.23: 编写一段程序，创建一个含有 10 个整数的 vector 对象，然后使用迭代器将所有元素的值都变成原来的两倍。输出 vector 对象的内容，检验程序是否正确。

3.4.2 迭代器运算



迭代器的递增运算令迭代器每次移动一个元素，所有的标准库容器都有支持递增运算的迭代器。类似的，也能用==和!=对任意标准库类型的两个有效迭代器（参见 3.4 节，第 95 页）进行比较。

string 和 vector 的迭代器提供了更多额外的运算符，一方面可使得迭代器的每次移动跨过多个元素，另外也支持迭代器进行关系运算。所有这些运算被称作**迭代器运算**（iterator arithmetic），其细节由表 3.7 列出。

表 3.7: vector 和 string 迭代器支持的运算	
iter + n	迭代器加上一个整数值仍得一个迭代器，迭代器指示的新位置与原来相比向前移动了若干个元素。结果迭代器或者指示容器内的一个元素，或者指示容器尾元素的下一位置
iter - n	迭代器减去一个整数值仍得一个迭代器，迭代器指示的新位置与原来相比向后移动了若干个元素。结果迭代器或者指示容器内的一个元素，或者指示容器尾元素的下一位置
iter1 += n	迭代器加法的复合赋值语句，将 iter1 加 n 的结果赋给 iter1
iter1 -= n	迭代器减法的复合赋值语句，将 iter1 减 n 的结果赋给 iter1
iter1 - iter2	两个迭代器相减的结果是它们之间的距离，也就是说，将运算符右侧的迭代器向前移动差值个元素后将得到左侧的迭代器。参与运算的两个迭代器必须指向的是同一个容器中的元素或者尾元素的下一位置
>, >=, <, <=	迭代器的关系运算符，如果某迭代器指向的容器位置在另一个迭代器所指位置之前，则说前者小于后者。参与运算的两个迭代器必须指向的是同一个容器中的元素或者尾元素的下一位置

迭代器的算术运算

可以令迭代器和一个整数值相加（或相减），其返回值是向前（或向后）移动了若干个位置的迭代器。执行这样的操作时，结果迭代器或者指示原 vector 对象（或 string 对象）内的一个元素，或者指示原 vector 对象（或 string 对象）尾元素的下一位置。

举个例子，下面的代码得到一个迭代器，它指向某 vector 对象中间位置的元素：

```
// 计算得到最接近 vi 中间元素的一个迭代器
auto mid = vi.begin() + vi.size() / 2;
```

如果 vi 有 20 个元素，vi.size()/2 得 10，此例中即令 mid 等于 vi.begin()+10。已知下标从 0 开始，则迭代器所指的元素是 vi[10]，也就是从首元素开始向前相隔 10 个位置的那个元素。

对于 string 或 vector 的迭代器来说，除了判断是否相等，还能使用关系运算符(<、<=、>、>=) 对其进行比较。参与比较的两个迭代器必须合法而且指向的是同一个容器的元素（或者尾元素的下一位置）。例如，假设 it 和 mid 是同一个 vector 对象的两个迭代器，可以用下面的代码来比较它们所指的位置孰前孰后：

```
if (it < mid)
    // 处理 vi 前半部分的元素
```

112

只要两个迭代器指向的是同一个容器中的元素或者尾元素的下一位置，就能将其相减，所得结果是两个迭代器的距离。所谓距离指的是右侧的迭代器向前移动多少位置就能追上左侧的迭代器，其类型是名为 **difference_type** 的带符号整型数。string 和 vector 都定义了 difference_type，因为这个距离可正可负，所以 difference_type 是带符号类型的。

使用迭代器运算

使用迭代器运算的一个经典算法是二分搜索。二分搜索从有序序列中寻找某个给定的值。二分搜索从序列中间的位置开始搜索，如果中间位置的元素正好就是要找的元素，搜索完成；如果不是，假如该元素小于要找的元素，则在序列的后半部分继续搜索；假如该元素大于要找的元素，则在序列的前半部分继续搜索。在缩小的范围中计算一个新的中间元素并重复之前的过程，直至最终找到目标或者没有元素可供继续搜索。

下面的程序使用迭代器完成了二分搜索：

```
// text 必须是有序的
// beg 和 end 表示我们搜索的范围
auto beg = text.begin(), end = text.end();
auto mid = text.begin() + (end - beg) / 2; // 初始状态下的中间点
// 当还有元素尚未检查并且我们还没有找到 sought 时执行循环
while (mid != end && *mid != sought) {
    if (sought < *mid) // 我们要找的元素在前半部分吗？
        end = mid; // 如果是，调整搜索范围使得忽略掉后半部分
    else // 我们要找的元素在后半部分
        beg = mid + 1; // 在 mid 之后寻找
    mid = beg + (end - beg) / 2; // 新的中间点
}
```

程序的一开始定义了三个迭代器：beg 指向搜索范围内的第一个元素、end 指向尾元素的下一位置、mid 指向中间的那个元素。初始状态下，搜索范围是名为 text 的 vector<string> 的全部范围。

循环部分先检查搜索范围是否为空，如果 mid 和 end 的当前值相等，说明已经找遍了所有元素。此时条件不满足，循环终止。当搜索范围不为空时，可知 mid 指向了某个元素，检查该元素是否就是我们所要搜索的，如果是，也终止循环。

当进入到循环体内部后, 程序通过某种规则移动 `beg` 或者 `end` 来缩小搜索的范围。如果 `mid` 所指的元素比要找的元素 `sought` 大, 可推测若 `text` 含有 `sought`, 则必出现在 `mid` 所指元素的前面。此时, 可以忽略 `mid` 后面的元素不再查找, 并把 `mid` 赋给 `end` 即可。另一种情况, 如果 `*mid` 比 `sought` 小, 则要找的元素必出现在 `mid` 所指元素的后面。此时, 通过令 `beg` 指向 `mid` 的下一个位置即可改变搜索范围。因为已经验证过 `mid` 不是我们要找的对象, 所以在接下来的搜索中不必考虑它。

循环过程终止时, `mid` 或者等于 `end` 或者指向要找的元素。如果 `mid` 等于 `end`, 说明 `text` 中没有我们要找的元素。

3.4.2 节练习

113

练习 3.24: 请使用迭代器重做 3.3.3 节 (第 94 页) 的最后一个练习。

练习 3.25: 3.3.3 节 (第 93 页) 划分分数段的程序是使用下标运算符实现的, 请利用迭代器改写该程序并实现完全相同的功能。

练习 3.26: 在 100 页的二分搜索程序中, 为什么用的是 `mid = beg + (end - beg) / 2`, 而非 `mid = (beg + end) / 2`?

3.5 数组

数组是一种类似于标准库类型 `vector` (参见 3.3 节, 第 86 页) 的数据结构, 但是在性能和灵活性的权衡上又与 `vector` 有所不同。与 `vector` 相似的地方是, 数组也是存放类型相同的对象的容器, 这些对象本身没有名字, 需要通过其所在位置访问。与 `vector` 不同的地方是, 数组的大小确定不变, 不能随意向数组中增加元素。因为数组的大小固定, 因此对某些特殊的应用来说程序的运行时性能较好, 但是相应地也损失了一些灵活性。



如果不清楚元素的确切个数, 请使用 `vector`。

3.5.1 定义和初始化内置数组

数组是一种复合类型 (参见 2.3 节, 第 45 页)。数组的声明形如 `a[d]`, 其中 `a` 是数组的名字, `d` 是数组的维度。维度说明了数组中元素的个数, 因此必须大于 0。数组中元素的个数也属于数组类型的一部分, 编译的时候维度应该是已知的。也就是说, 维度必须是一个常量表达式 (参见 2.4.4 节, 第 58 页):

```
unsigned cnt = 42;           // 不是常量表达式
constexpr unsigned sz = 42; // 常量表达式, 关于 constexpr, 参见 2.4.4 节 (第 59 页)
int arr[10];                 // 含有 10 个整数的数组
int *parr[sz];               // 含有 42 个整型指针的数组
string bad[cnt];             // 错误: cnt 不是常量表达式
string strs[get_size()];     // 当 get_size 是 constexpr 时正确; 否则错误
```

默认情况下, 数组的元素被默认初始化 (参见 2.2.1 节, 第 40 页)。



和内置类型的变量一样, 如果在函数内部定义了某种内置类型的数组, 那么默认初始化会令数组含有未定义的值。